Assignment-19 (Lab-19.2)

Course : AI Assisted Coding

Topic: Code Translation

Student Details

Name :Batti Chandan Singh

Hall Ticket :2303A51515

Batch :22

Task-01

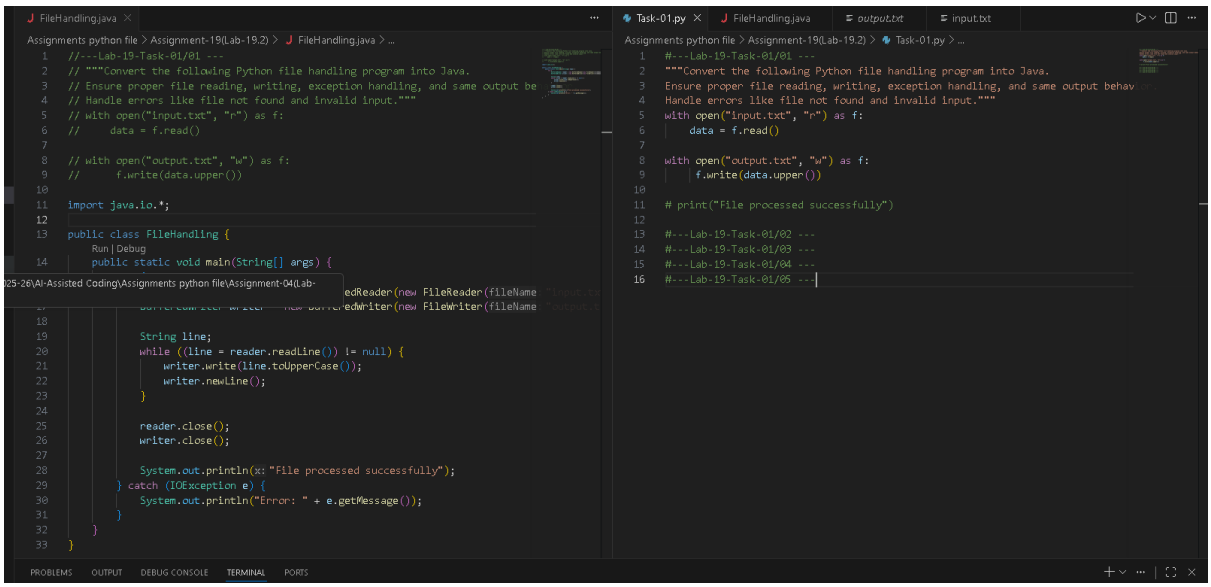
Final Optimal Prompt

Convert the following Python file handling program into Java.

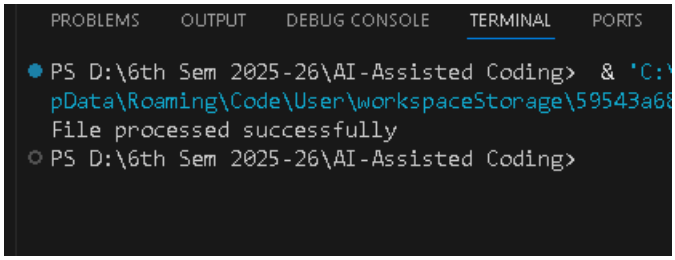
Ensure proper file reading, writing, exception handling, and same output behavior.

Handle errors like file not found and invalid input.

Code Screenshot



Output Screenshot



Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task converts a Python file handling program into Java. The Python version uses simple file operations with open() for reading and writing. In Java, file handling is done using BufferedReader and BufferedWriter, which require explicit exception handling. The Java program reads the file line by line, converts text to uppercase, and writes it to another file. Try-catch blocks handle errors like missing files or read/write issues. Both programs produce the same output and file content. This task highlights differences in syntax and complexity between Python and Java, while maintaining identical functionality.

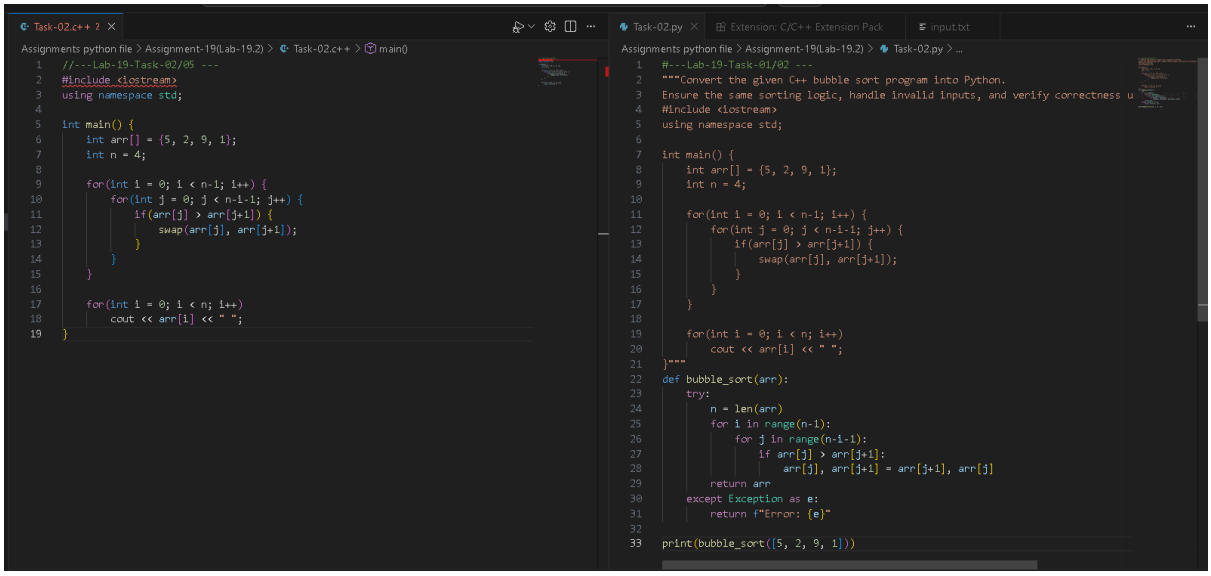
Task-02

Final Optimal Prompt

Convert the given C++ bubble sort program into Python.

Ensure the same sorting logic, handle invalid inputs, and verify correctness using test cases.

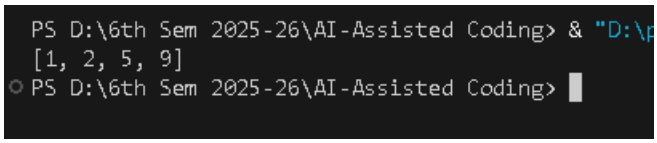
Code Screenshot



```
//---Lab-19-Task-02/05 ---
#include <iostream>
using namespace std;

int main() {
    int arr[] = {5, 2, 9, 1};
    int n = 4;

    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }

    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";
}
```

```
#---Lab-19-Task-01/02 ---
"""Convert the given C++ bubble sort program into Python.
Ensure the same sorting logic, handle invalid inputs, and verify correctness u
#include <iostream>
using namespace std;

int main() {
    int arr[] = {5, 2, 9, 1};
    int n = 4;

    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }

    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";
}"""
def bubble_sort(arr):
    try:
        n = len(arr)
        for i in range(n-1):
            for j in range(n-i-1):
                if arr[j] > arr[j+1]:
                    arr[j], arr[j+1] = arr[j+1], arr[j]
        return arr
    except Exception as e:
        return f"Error: {e}"

print(bubble_sort([5, 2, 9, 1]))
```

Output Screenshot



```
PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\p
[1, 2, 5, 9]
PS D:\6th Sem 2025-26\AI-Assisted Coding>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task translates a C++ bubble sort program into Python. The logic remains the same, using nested loops to compare and swap elements. Python simplifies the syntax by allowing tuple swapping instead of a separate swap function. Error handling is added using try-except to handle unexpected inputs. The function returns the sorted list, making it reusable. Both programs produce identical sorted output for the same input. This task demonstrates how algorithms can be translated across languages while preserving logic. It also highlights Python’s simplicity compared to C++ in terms of syntax and memory handling.

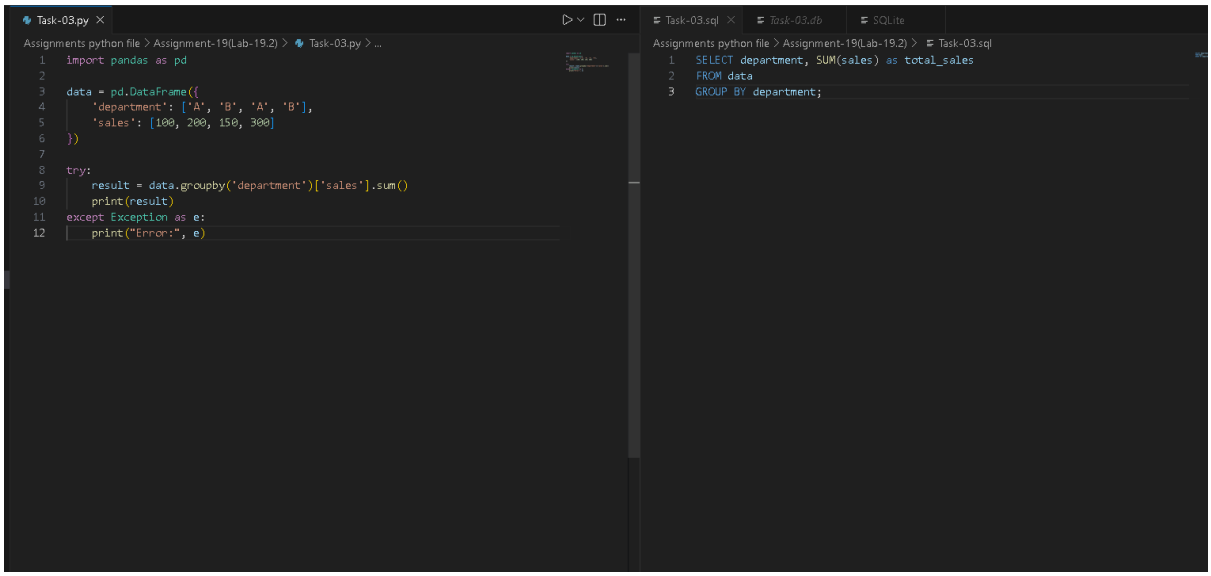
Task-03

Final Optimal Prompt

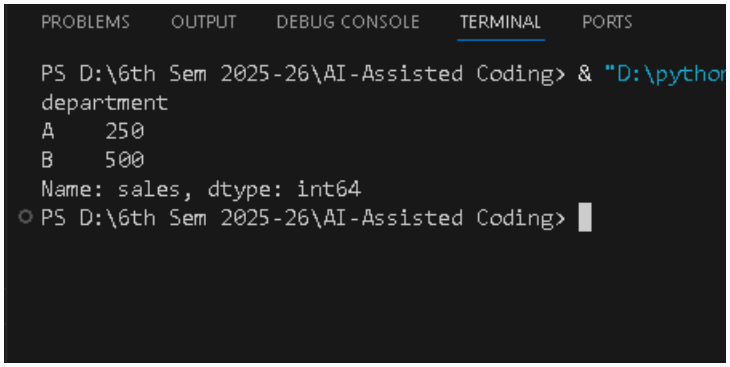
Convert the given SQL query with GROUP BY and aggregation into Pandas code.

Ensure equivalent output and handle missing or invalid data.

Code Screenshot



Output Screenshot



Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task converts an SQL aggregation query into a Pandas operation. The SQL query uses GROUP BY and SUM to calculate total sales per department. In Pandas, this is achieved using groupby() and sum() functions. The DataFrame structure represents table-like data similar to SQL tables. Error handling is added to manage issues like missing columns or invalid data types. Both SQL and Pandas produce identical results, ensuring correctness. This task highlights how data analysis can be performed in Python using Pandas instead of SQL. It also shows how AI helps translate queries across different technologies efficiently.

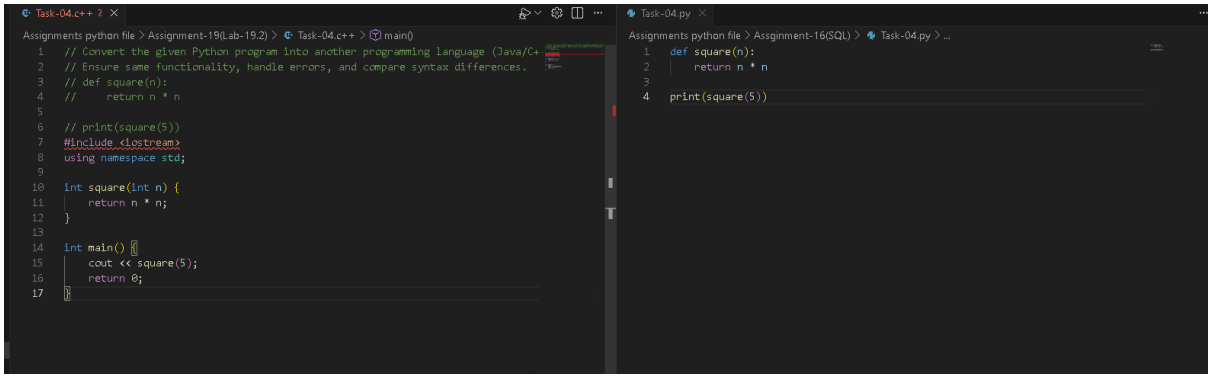
Task-04

Final Optimal Prompt

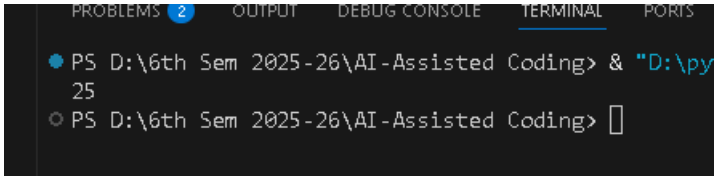
Convert the given Python program into another programming language (Java/C++).

Ensure same functionality, handle errors, and compare syntax differences.

Code Screenshot



Output Screenshot



Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task demonstrates translation of a simple Python program into C++. The function calculates the square of a number. In Python, the function is concise and does not require type declarations. In C++, data types and a main function are required. Both implementations produce the same output. Error handling can be added in C++ using input validation if needed. This task shows differences in syntax, structure, and compilation requirements between languages. It highlights how AI tools help developers quickly convert code across languages while maintaining functionality and correctness in real-world applications.

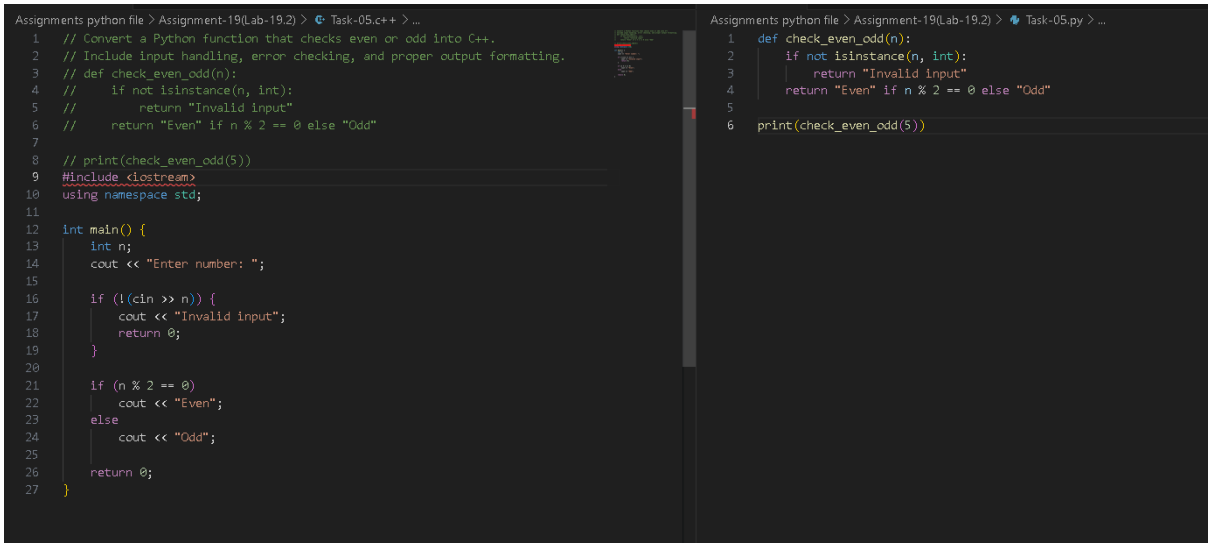
Task-05

Final Optimal Prompt

Convert a Python function that checks even or odd into C++.

Include input handling, error checking, and proper output formatting.

Code Screenshot



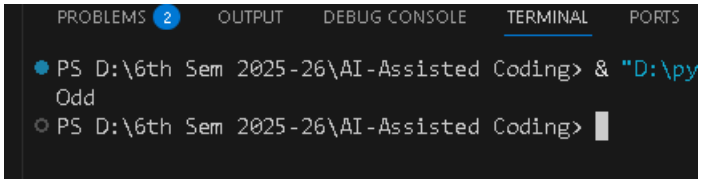
Assignments python file > Assignment-19(Lab-19.2) > Task-05.c++ > ...

```cpp
// Convert a Python function that checks even or odd into C++.
// Include input handling, error checking, and proper output formatting.
// def check_even_odd(n):
//     if not isinstance(n, int):
//         return "Invalid input"
//     return "Even" if n % 2 == 0 else "Odd"

// print(check_even_odd(5))
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number: ";

    if (!(cin >> n)) {
        cout << "Invalid input";
        return 0;
    }

    if (n % 2 == 0)
        cout << "Even";
    else
        cout << "Odd";

    return 0;
}
```

Assignments python file > Assignment-19(Lab-19.2) > Task-05.py > ...

```python
def check_even_odd(n):
    if not isinstance(n, int):
        return "Invalid input"
    return "Even" if n % 2 == 0 else "Odd"

print(check_even_odd(5))
```

Output Screenshot



```
PROBLEMS 2  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\py
Odd
PS D:\6th Sem 2025-26\AI-Assisted Coding>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task converts a Python function into a C++ program for checking whether a number is even or odd. Python uses dynamic typing and simple conditions, while C++ requires explicit input handling using cin. Error handling is implemented in C++ to detect invalid input using stream validation. Both programs follow the same logic using the modulus operator. The output remains consistent across both implementations. This task highlights the differences in syntax, type handling, and input validation between Python and C++. It also demonstrates how AI tools simplify the translation process while ensuring logical correctness and usability.